
UCI Control Target

Command Summary

Gideon Zweijtzer

All work Copyright © 2026 by
Gideon's Logic B.V.
All rights reserved.

Table of Contents

1. Introduction	3
1.1. Context.....	3
1.2. Purpose of this document.....	3
2. Commands.....	4
2.1. CTRL_CMD_IDENTIFY (0x01)	4
2.2. CTRL_CMD_FINISH_CAPTURE (0x03).....	4
2.3. CTRL_CMD_FREEZE (0x05)	4
2.4. CTRL_CMD_REBOOT (0x06).....	4
2.5. CTRL_CMD_LOAD_REU (0x08) / SAVE_REU (0x09)	5
2.6. CTRL_CMD_U64_SAVEMEM (0x0F).....	5
2.7. CTRL_CMD_DECODE_TRACK (0x11)	5
2.8. CTRL_CMD_EASYFLASH_ERASE (0x20).....	6
2.9. CTRL_CMD_GET_HWINFO (0x28) - DEPRECATED.....	6
2.10. CTRL_CMD_GET_DRVINFO (\$29)	7
2.11. CTRL_CMD_ENABLE_DRIVE_A (0x30)	7
2.12. CTRL_CMD_DISABLE_DRIVE_A (0x31)	7
2.13. CTRL_CMD_ENABLE_DRIVE_B (0x32)	7
2.14. CTRL_CMD_DISABLE_DRIVE_B (0x33)	7
2.15. CTRL_CMD_GET_DRIVE_A_POWER (0x34)	8
2.16. CTRL_CMD_GET_DRIVE_B_POWER (0x35)	8
2.17. CTRL_CMD_GET_MP3_RAMDISKINFO (0x40)	8
2.18. CTRL_CMD_GET_PALETTE (0x51).....	8
2.19. CTRL_CMD_SET_PALETTE (0x52)	8
2.20. CTRL_CMD_SET_PALETTE_COLOR (0x53).....	9
2.21. CTRL_CMD_RESET_PALETTE (0x54).....	9
2.22. Runtime and VPL file behavior.....	9
3. Palette programming examples	11
3.1. 6502 assembly: set one color, then reset.....	11
3.2. 6502 assembly: read all 16 colors	12
3.3. Commodore BASIC V2: set one color, then reset.....	13

1. Introduction

1.1. Context

The “**UCI Control Target**” provides a low-level interface to manage the hardware state of the Ultimate cartridge or Ultimate 64 machines and its emulated environment. This includes direct control over C64 execution (freeze/reboot), power management for emulated disk drives, and advanced utilities for REU and disk image handling.

The “UCI Control Target” is a target of the “Ultimate Command Interface” (UCI), and is thus accessible from the cartridge port, through some I/O registers. The document “Ultimate Command Interface – Register API” describes how commands are sent over this interface.

1.2. Purpose of this document

This document describes the commands that can be sent to this target, their expected parameters, and the data returned to the C64. These commands allow programs to interact with the Ultimate’s management engine without leaving the C64 environment.

2. Commands

In the Ultimate products and the Commodore 64 Ultimate, the “Control Target” is accessible through target \$04. This shall be the first byte of the command.

The following paragraphs describe each of the commands of “Control Target”.

2.1. CTRL_CMD_IDENTIFY (0x01)

Command format: \$04 \$01

The “Identify” command sends back an identification string, such as:

“CONTROL TARGET V1.1”. The user software can use this function to query which targets exist, or to obtain version information.

The status channel will report 00, OK, as this command cannot fail.

2.2. CTRL_CMD_FINISH_CAPTURE (0x03)

Command format: \$04 \$03

Description: Finalizes an active tape capture session and closes the associated file.

Status: 00, OK.

2.3. CTRL_CMD_FREEZE (0x05)

Command format: \$04 \$05

Description: Triggers a hardware freeze, equivalent to pressing the physical button on the Ultimate cartridge.

Status: 00, OK

Note that the command still needs to be acknowledged after the freeze has completed. This might be problematic when the user selects something else in the menu. To be fixed / addressed.

2.4. CTRL_CMD_REBOOT (0x06)

Command format: \$04 \$06

Description: Triggers a full system reboot the C64. It corresponds to the menu item “Reboot” in the actions menu. This command also re-initializes the utility cartridge settings to the stored configuration.

Response: None

Status: None

Because the C64 gets reset; the Ultimate Command Interface (UCI) gets reset as well. Therefore no response is to be expected.

2.5. CTRL_CMD_LOAD_REU (0x08) / SAVE_REU (0x09)

Command format: \$04 \$08 <FILENAME> or \$04 \$09 <FILENAME>

Description: Loads an REU image from storage into the Ultimate's REU memory, or saves the current REU contents to a file.

Response: A 4-byte little-endian value representing number of bytes transferred. Negative values indicate an error; see status reply.

Status:

- 00, OK
- 81, INVALID PARAMS
- 84, REU NOT ENABLED
- 85, REU FILE CANNOT BE OPENED.

2.6. CTRL_CMD_U64_SAVEMEM (0x0F)

Command format: \$04 \$0F <FILENAME>

This command is only valid on the Ultimate 64, Ultimate 64 Elite, Ultimate 64 Elite-II and the Commodore 64 Ultimate.

This command saves the entire RAM. It does not save any other state information. When the filename is omitted, it will save by default to "/temp/c64_memory.bin"

Status:

- 00, OK
- 87, DISK ERR: <error string>

2.7. CTRL_CMD_DECODE_TRACK (0x11)

Command format: \$04 \$11 <TRK> <MAX_SEC> <GCR_ADDR> <BIN_ADDR> <GCR_LEN>

Description: High-speed GCR to Binary sector conversion.

Parameters:

- TRK is the expected track number.
- MAX_SEC is the highest sector number expected.
- GCR_ADDR is the offset in REU memory where the GCR data is located, LSB first.
- BIN_ADDR is the offset in REU memory where the decoded binary data will be stored.
- GCR_LEN is the length of the offered GCR track in bytes, LSB first.

Response: 1 byte (actual sectors) followed by 2 bytes of error flags per sector.

Status:

- 00, OK
- 82, ERRORS ON TRACK.

2.8. CTRL_CMD_EASYFLASH_ERASE (0x20)

Command format: \$04 \$20 \$00 <BANK> <BASEADDR>

Description: The EasyFlash Erase command emulates the sector-erase function of an EasyFlash cartridge. This command erases a 64 KiB sector of the emulated 1 MiB Flash chip.

Parameters:

- Sub-command: Must be \$00 (Erase)
- Bank: The selected 16 KiB bank (bits 3-5 are used to determine the 64 KiB sector)
- Base Address: The high byte of the C64 address. The Ultimate uses this to determine if the Low (\$8000-\$9FFF) or High (\$A000-\$BFFF) ROM area is targeted.

The Ultimate clears 8 banks of 8 KiB (64 KiB total) by setting all bytes to \$FF.

Status:

- 00, OK
- 81, INVALID PARAMS

2.9. CTRL_CMD_GET_HWINFO (0x28) - DEPRECATED

Command format: \$04 \$28 <SUB_CMD>

Description: Queries hardware information. This command has two sub-commands, which violates the UCI concept. From firmware 3.15, when the sub command is not given, it defaults to returning the hardware model name in ASCII format.

- **Sub Command \$00:** Returns the hardware model name (e.g., "ULTIMATE 64").
- **Sub Command \$01:** Returns SID configuration, including base addresses and enable masks:
 - First byte contains the number of SID info frames follow.
 - Each SID info frame consists of 5 bytes:
 - Primary base address (2 bytes, LSB first)
 - Secondary base address (2 bytes, MSB first)
 - Type indicator, but unclear what it means; it seems not to match the actual implementation of SID control.

2.10. CTRL_CMD_GET_DRVINFO (\$29)

Command format: \$04 \$29 <EFFECTIVE_ADDR_FLAG>

Description: This command returns the devices that the Ultimate

Response: Byte 0 is drive count, followed by 3-byte groups for each drive: [Type] [IEC Bus Address] [Power State]. The drive types are as follows:

Type	Value
1541	\$00
1571	\$01
1581	\$02
Undecided	\$03
Software IEC	\$0F
Printer	\$50

The power state is either \$00 or \$01.

Status: 00,OK

2.11. CTRL_CMD_ENABLE_DRIVE_A (0x30)

Command format: \$04 \$30

Description: Controls the power state of emulated drive A: This command turns the drive ON.

Status: 00,OK

2.12. CTRL_CMD_DISABLE_DRIVE_A (0x31)

Command format: \$04 \$31

Description: Controls the power state of emulated drive A: This command turns the drive OFF.

Status: 00,OK

2.13. CTRL_CMD_ENABLE_DRIVE_B (0x32)

Command format: \$04 \$32

Description: Controls the power state of emulated drive B: This command turns the drive ON.

Status: 00,OK

2.14. CTRL_CMD_DISABLE_DRIVE_B (0x33)

Command format: \$04 \$33

Description: Controls the power state of emulated drive B: This command turns the drive OFF.

Status: 00,OK

2.15. CTRL_CMD_GET_DRIVE_A_POWER (0x34)

Command format: \$04 \$34

Description: With this command the power state of drive A can be queried.

Response: "off" or "on "

Status: 00,OK

2.16. CTRL_CMD_GET_DRIVE_B_POWER (0x35)

Command format: \$04 \$35

Description: With this command the power state of drive B can be queried.

Response: "off" or "on "

Status: 00,OK

2.17. CTRL_CMD_GET_MP3_RAMDISKINFO (0x40)

Command format: \$04 \$40

Description: This is a command specifically made for Geos MegaPatch 3. It returns the configuration of the RAM disks found in the REU image.

Response: An 8-byte status block (2 bytes per drive) indicating RAM disk types (1541, 1571, 1581, or Native) and their sizes. The first of the two bytes indicates the type (0x41, 0x71, 0x81 or 0xDD). For native partitions, the second byte indicates the size expressed in blocks of 64 KiB, otherwise it's zero.

Status: 00,OK

2.18. CTRL_CMD_GET_PALETTE (0x51)

Command format: \$04 \$51

Availability: Ultimate 64, Ultimate 64 Elite, Ultimate 64-II, Ultimate 64 Elite-II, and Commodore 64 Ultimate.

Description: Returns the currently applied runtime palette. The response is 48 bytes: 16 consecutive RGB triples in C64 color-number order (0 through 15). Each component is an unsigned 8-bit value. For example, response bytes 45, 46, and 47 are the red, green, and blue components of color 15.

Response: [R0 G0 B0] ... [R15 G15 B15]

Status: 00,OK. A payload or any command length other than two bytes returns 81,INVALID PARAMS.

On other Ultimate products this command is not implemented and returns 21,UNKNOWN COMMAND.

2.19. CTRL_CMD_SET_PALETTE (0x52)

Command format: \$04 \$52 [R0 G0 B0] ... [R15 G15 B15]

Availability: Ultimate 64 family only, as listed for GET_PALETTE.

Description: Replaces all 16 colors of the runtime palette. The command must contain exactly 48 RGB bytes after the target and command bytes, for a total command length of 50 bytes.

Response: None.

Status: 00,OK, or 81,INVALID PARAMS when the command length is not exactly 50 bytes. Invalid commands do not change the palette.

2.20. CTRL_CMD_SET_PALETTE_COLOR (0x53)

Command format: \$04 \$53 [INDEX] [R] [G] [B]

Availability: Ultimate 64 family only, as listed for GET_PALETTE.

Description: Replaces one runtime color without rewriting the other 15 colors. INDEX is the C64 color number and must be in the range 0 through 15. The command must be exactly six bytes long.

Response: None.

Status: 00,OK, or 81,INVALID PARAMS for an invalid index or command length. Invalid commands do not change the palette.

2.21. CTRL_CMD_RESET_PALETTE (0x54)

Command format: \$04 \$54

Availability: Ultimate 64 family only, as listed for GET_PALETTE.

Description: Restores the runtime palette to the firmware's built-in C64 default colors. No RGB payload is required.

Response: None.

Status: 00,OK. A payload or any command length other than two bytes returns 81,INVALID PARAMS without changing the palette.

2.22. Runtime and VPL file behavior

The four palette commands change runtime video output only. They do not write flash configuration and do not select, create, or overwrite a VPL file. A later palette change from the configuration UI, applying a VPL file, or rebooting the Ultimate may replace the runtime palette.

A C64 program can load and save VPL files through the existing Ultimate-DOS UCI target (\$01 or \$02):

1. To load: DOS_CMD_OPEN_FILE (\$02) with attribute \$01, DOS_CMD_READ_DATA (\$04), then DOS_CMD_CLOSE_FILE (\$03). Parse 16 RGB lines and send them with SET_PALETTE.
2. To save or overwrite: DOS_CMD_OPEN_FILE (\$02) with attribute \$0E, one or more DOS_CMD_WRITE_DATA (\$05) commands, then DOS_CMD_CLOSE_FILE (\$03).

A VPL file contains 16 non-empty RGB lines in C64 color-number order. Components are hexadecimal. Blank lines and text after # are ignored. This is the built-in default palette in VPL form:

00 00 00 # 0 black
F7 F7 F7 # 1 white
8D 2F 34 # 2 red
6A D4 CD # 3 cyan
98 35 A4 # 4 purple
4C B4 42 # 5 green
2C 29 B1 # 6 blue
EF EF 5D # 7 yellow
98 4E 20 # 8 orange
5B 38 00 # 9 brown
D1 67 6D # 10 light red
4A 4A 4A # 11 dark gray
7B 7B 7B # 12 gray
9F EF 93 # 13 light green
6D 6A EF # 14 light blue
B2 B2 B2 # 15 light gray

3. Palette programming examples

The Command Interface setting must be enabled before these examples run. The transport and register meanings are defined in *Ultimate Command Interface - Register API*.

3.1. 6502 assembly: set one color, then reset

This complete example prints a banner using color 15, changes that color from light gray to red, waits for a key, and restores the built-in palette. Assemble it at \$C000 and start it with SYS 49152.

Production code should also read and check the status bytes at \$DF1F; successful commands return the ASCII string 00,OK.

* = \$C000

UCI_CONTROL = \$DF1C
UCI_COMMAND = \$DF1D

PUSH_CMD = \$01
DATA_ACCEPT = \$02
STATE_MASK = \$30
STATE_BUSY = \$10
STATE_LAST = \$20

start:

```
lda #$93      ; clear screen
jsr $FFD2     ; KERNAL CHROUT
lda #$0f
sta $0286     ; current text color = color 15
ldx #$00
```

show: lda banner,x

```
beq change
jsr $FFD2
inx
bne show
```

change: jsr set_light_gray_red

```
key: jsr $FFE4 ; KERNAL GETIN
beq key
jsr reset_palette
rts
```

banner: .text "COLOR 15: RED - PRESS A KEY", 0

set_light_gray_red:

```
lda #$04      ; Control target
sta UCI_COMMAND
lda #$53      ; SET_PALETTE_COLOR
sta UCI_COMMAND
lda #$0f      ; color 15
sta UCI_COMMAND
```

```
lda #$ff      ; red
sta UCI_COMMAND
lda #$00      ; green, blue
sta UCI_COMMAND
sta UCI_COMMAND
jsr send_and_accept
rts
```

reset_palette:

```
lda #$04      ; Control target
sta UCI_COMMAND
lda #$54      ; RESET_PALETTE
sta UCI_COMMAND
jsr send_and_accept
rts
```

send_and_accept:

```
lda #PUSH_CMD
sta UCI_CONTROL
```

wait: lda UCI_CONTROL

```
and #STATE_MASK
cmp #STATE_BUSY
beq wait
cmp #STATE_LAST
bne protocol_error
lda #DATA_ACCEPT
sta UCI_CONTROL
rts
```

protocol_error:

```
; Application-specific error handling goes here.
rts
```

3.2. 6502 assembly: read all 16 colors

After STATE_LAST is reached, bit 7 of \$DF1C indicates that another response byte is available at \$DF1E.

UCI_RESPONSE = \$DF1E

palette = \$C000 ; 48-byte application buffer

get_palette:

```
lda #$04
sta UCI_COMMAND
lda #$51      ; GET_PALETTE
sta UCI_COMMAND
lda #PUSH_CMD
sta UCI_CONTROL
```

wait2: lda UCI_CONTROL

```

    and #STATE_MASK
    cmp #STATE_BUSY
    beq wait2
    cmp #STATE_LAST
    bne protocol_error
    ldx #$00
read: lda UCI_CONTROL
    bpl done      ; bit 7 clear: response queue empty
    lda UCI_RESPONSE
    sta palette,x
    inx
    bne read
done: cpx #48
    bne protocol_error
    lda #DATA_ACCEPT
    sta UCI_CONTROL
    rts

```

3.3. Commodore BASIC V2: set one color, then reset

Stock BASIC has no built-in UCI statement, but it can access the UCI registers with POKE and PEEK. This complete example prints a banner in color 15, changes it to red, waits for a key, and then uses RESET_PALETTE to return it to light gray.

```

10 C=57116:D=57117:T=57119
20 IF PEEK(D)<>201 THEN PRINT "UCI NOT AVAILABLE":END
25 PRINT CHR$(147):POKE 646,15:PRINT "COLOR 15: RED - PRESS A KEY"
30 N=6:GOSUB 200
50 GET K$:IF K$="" THEN 50
60 N=0:GOSUB 200
70 PRINT "DEFAULT PALETTE RESTORED":END
100 DATA 4,83,15,255,0,0
200 IF (PEEK(C) AND 48)<>0 THEN PRINT "UCI BUSY":END
210 IF N=0 THEN POKE D,4:POKE D,84:GOTO 230
220 FOR I=1 TO N:READ B:POKE D,B:NEXT
230 POKE C,1
240 S=PEEK(C) AND 48:IF S=16 THEN 240
250 IF S<>32 THEN PRINT "UCI PROTOCOL ERROR":END
260 A$=""
270 IF (PEEK(C) AND 64)=0 THEN 290
280 A$=A$+CHR$(PEEK(T)):GOTO 270
290 POKE C,2:PRINT A$:RETURN

```

The repository also contains software/6502/cmd_test_rom.tas, an old test cartridge that installs an @ BASIC statement. With that test ROM active, the single-color command can be written as @\$04\$53\$0F\$FF\$00\$00". The @ extension is not installed at the normal production BASIC prompt, so portable programs should use the register protocol above or install their own wedge.